

Multi-Agent Implementation — Presentation Script

Read this word-for-word when asked about multi-agent architecture

HOW TO USE THIS SCRIPT

- SAY: → Read these words out loud, exactly as written
- [ACTION] → Do this on your screen before speaking the next line
- [POINT TO] → Move your mouse cursor to this area so audience eyes follow
- [PAUSE] → Stop talking for 2–3 seconds. Let them look. Silence is powerful.
- Everything in () is a note for you — do NOT read it out loud

BEFORE YOU START — Setup Checklist

(Do this before you open the document on screen)

- ☐ Open MULTI-AGENT-IMPLEMENTATION.pdf on your screen — scroll to the top
- ☐ Have VS Code open with the .claude/workflows/ folder visible in the sidebar — minimised in taskbar
- ☐ This script open on your phone or a second monitor

OPENING — When Someone Asks "Did You Use Multi-Agent?"

(This is your trigger. Someone from the audience just asked. Take a breath. Don't rush.)

SAY:

"Yes — and I want to show you exactly how. Give me one minute to pull up the architecture document."

[ACTION: Open MULTI-AGENT-IMPLEMENTATION.pdf — share your screen. Make sure it is scrolled to the very top.]

SAY:

"So — multi-agent in this project works at two levels. And I want to be precise about this because it is an important distinction."

[POINT TO: the two-layer box diagram at the top of the document]

SAY:

"Layer one is the development layer — this is where the multi-agent system lives. AI agents were used to design, implement, test, and review the code that you just saw in the demo. Every feature was built through an orchestrated pipeline of specialist agents handing work to each other.

Layer two is the runtime layer — that is the SupportIQ platform itself. Five AI endpoints, each making a single call to EPAM DIAL. The platform is the output of the multi-agent system, not a multi-agent system itself."

[PAUSE — 3 seconds.]

SAY:

"Let me walk you through Layer one in detail."

ACT 1 — The Six Specialist Agents (~2 minutes)

[ACTION: Scroll down to the "Specialist Agents" section — the table with 6 rows]

SAY:

"Six agents were configured for this project. Each one has a dedicated system prompt — think of it as a job description. Each agent knows exactly what it is responsible for and nothing else."

[POINT TO: the first row — taskflow-architect]

SAY:

"The architect agent. Its only job is to read requirements and the current codebase, and produce a precise implementation plan — which file changes, which fields to add, which validation rules to apply. It does not write code. It plans."

[POINT TO: taskflow-reviewer row]

SAY:

"The reviewer agent. It checks every changed file against twenty-plus Spring Boot conventions. It returns structured findings with a severity rating — HIGH, MEDIUM, or LOW. High severity means the code does not merge until it is fixed."

[POINT TO: taskflow-tester row]

SAY:

"The tester agent. It knows this project's exact test patterns — @WebMvcTest, @MockBean, MockMvc assertions. It writes tests that match the acceptance criteria from the original requirement, one test per criterion."

[POINT TO: support-analyst-agent, support-triage-agent, demo-guide-agent rows]

SAY:

"And then three domain specialists for SupportIQ specifically. The support analyst designed the AI prompts and JSON schema contracts you saw in the demo. The triage agent designed the urgency ranking logic. The demo guide agent produced the management presentation scripts."

[PAUSE — 2 seconds.]

SAY:

"None of these agents know about each other directly. The orchestration workflow is what connects them."

ACT 2 — Feature Implementation Workflow (~4 minutes)

(This is the most technically impressive part — slow down here)

[ACTION: Scroll down to "Workflow 1 — Feature Implementation" and the 6-phase flow diagram]

SAY:

"This is the workflow that built SupportIQ. Every feature — every endpoint, every validation rule, every test — went through this six-phase pipeline. Let me walk you through it."

[POINT TO: PHASE 1 — RESEARCH box]

SAY:

"Phase one — Research. Two agents run simultaneously. Agent A connects to GitHub via the MCP server and reads the issue — title, requirements, acceptance criteria — and returns it as structured JSON. Agent B reads the current Java source files at the same moment — what fields exist, what imports are needed."

These two agents never talk to each other. They run in parallel and their outputs are merged by the orchestrator before the next phase begins."

[PAUSE — 2 seconds.]

[POINT TO: PHASE 2 — PLAN box]

SAY:

"Phase two — the architect agent receives both outputs. The issue requirements AND the current codebase state. It produces a plan — a JSON object that lists exactly what changes in each file. Entity changes. DTO changes. Mapper changes. Repository changes. Test changes. This plan becomes the input for every agent in the next phase."

[PAUSE — 2 seconds.]

[POINT TO: PHASE 3 — IMPLEMENT box with 3 parallel agents]

SAY:

"Phase three — three agents implement in parallel. One agent edits the entity file. A second agent edits the request DTO. A third agent edits the response DTO. All three are working at the same time, on different files, each receiving only the part of the architect's plan that is relevant to them."

[PAUSE — 3 seconds. Let them absorb the parallelism.]

SAY:

"This is where agent communication matters. The architect's plan is the shared contract. Each implementer agent receives the plan as input, makes its changes, and the orchestrator waits for all three to complete before continuing."

[POINT TO: PHASE 4 — INTEGRATE box]

SAY:

"Phase four — integration. The mapper and repository agents run in parallel. They run after phase three because they depend on the updated DTOs. This is a deliberate sequencing decision — the workflow enforces dependency order."

[POINT TO: PHASE 5 — TEST box]

SAY:

"Phase five — the tester agent runs. It receives the acceptance criteria from the original GitHub issue — the exact same text the business owner wrote — and turns each criterion into a test assertion. The test does not pass unless the acceptance criterion is satisfied."

[POINT TO: PHASE 6 — REVIEW box]

SAY:

"Phase six — the reviewer agent. It reads all the changed files and checks every requirement from the issue. Was every field added? Does every criterion have a test? Any @Autowired that should be constructor injection? It returns a structured result — readyToCommit: true or false. If false, the developer sees exactly what to fix."

[PAUSE — 4 seconds.]

SAY:

"From GitHub issue to production-ready code — six phases, nine agents total, running in a structured pipeline where each agent's output is the next agent's input. The developer's job was to review the result and commit."

ACT 3 — Code Review Workflow (~2 minutes)

[ACTION: Scroll down to "Workflow 2 — Code Review"]

SAY:

"The second workflow is the code review. Before anything was committed to the feature branch, four specialist reviewer agents ran simultaneously on the same diff."

[POINT TO: the 4 parallel agents in PHASE 2]

SAY:

"Agent one checks Spring Boot conventions — constructor injection, @Transactional, @Slf4j. Agent two checks validation — exact message strings, @Valid annotations. Agent three checks test coverage — are all new code paths covered? Agent four checks API design — HTTP status codes, path naming, OpenAPI annotations.

Four different lenses, four different agents, all examining the same code at the same time."

[POINT TO: PHASE 3 — SYNTHESIZE]

SAY:

"The synthesizer agent then receives all four finding sets, merges them, deduplicates cross-dimension findings, and returns a single verdict — APPROVED or NEEDS CHANGES — with a prioritised list of what to fix."

[PAUSE — 2 seconds.]

SAY:

"This is the equivalent of having four senior engineers review your code simultaneously, each expert in a different dimension, and a tech lead summarise their findings in one report."

ACT 4 — Agent Communication Pattern (~2 minutes)

(This is the technical heart — architects will ask about this)

[ACTION: Scroll down to "How Agents Talk to Each Other" section]

SAY:

"The question I usually get here is — how exactly do the agents communicate? Let me show you."

[POINT TO: the JSON handoff diagram]

SAY:

"The agents do not send messages to each other in real time. They communicate through structured data handoffs. Agent A produces a typed JSON object. The orchestrator passes that exact JSON as input into Agent B's prompt."

[POINT TO: the first JSON block — issue requirements]

SAY:

"Look at this. The MCP reader agent returns the issue requirements as a JSON array. That array is directly injected into the architect agent's prompt — not summarised, not paraphrased, the raw structured output. The architect reads it and produces its own JSON — a plan per file."

[POINT TO: the second JSON block — architect plan]

SAY:

"That plan is then split — the entity changes go to the entity implementer, the DTO changes go to the DTO implementer, the test changes go to the tester. Each agent sees only what it needs. No agent has context it does not need."

[PAUSE — 3 seconds.]

SAY:

"This is the key design principle. The workflow is the coordinator. The agents are the specialists. The data flowing between them is typed and validated. This is not a chat between agents — it is a structured production pipeline where each step has a defined input schema and a defined output schema."

ACT 5 — Support AI Insights Workflow (~1.5 minutes)

[ACTION: Scroll down to "Workflow 3 — Support AI Insights"]

SAY:

"The third workflow is different — this one runs against the live SupportIQ platform. It is the analytical layer on top of the product."

[POINT TO: the 4 parallel analysts in PHASE 2]

SAY:

"Four agents receive the same live queue data — the same ticket counts, sentiment scores, escalation rates — but each analyses a completely different dimension. Queue health, churn risk, sentiment trends, and issue patterns."

"Each agent produces a score out of one hundred and a list of findings. The synthesizer then receives all four scores and all four finding sets and writes an executive briefing — the kind of report a VP of Customer Success would read before a Monday morning meeting."

[PAUSE — 2 seconds.]

SAY:

"This workflow is what I ran when building the management presentation you saw earlier. The talking points, the risk numbers, the recommendations — those came from this multi-agent analysis, not from me writing them manually."

CLOSING — The Architecture Takeaway (~1 minute)

[ACTION: Scroll to the bottom — "Key Takeaway for Architects" section]

SAY:

"Let me leave you with the architectural principle behind all of this."

[POINT TO: the last paragraph]

SAY:

"Every agent in this system has four things: a defined role, a typed input, a typed output, and a single responsibility. The orchestration — what runs in parallel, what waits, what data flows forward — is deterministic code, not model-driven. The models handle reasoning within their scope. The workflow handles coordination across scopes."

"This separation is what makes the system reliable. If an agent produces bad output, you fix that agent's prompt. You do not touch the orchestration. If the orchestration needs a new phase, you add it without changing any agent."

"And the result — one hundred and three tests, two thousand lines of production Java, five AI endpoints, a full management dashboard — all built in eight days of an AI-native engineering sprint."

[PAUSE — 4 seconds.]

SAY:

"The question is not whether multi-agent AI can build production software. I just showed you that it already has."

[PAUSE — 5 seconds. Do not break the silence. Let them respond first.]

QUESTION & ANSWER — Ready Responses

Q: "How did the agents know the project's conventions?"

SAY:

"Each specialist agent has a custom system prompt — essentially a detailed job description. The taskflow-reviewer's prompt lists every convention: constructor injection only, @MockBean not @Mock, exact validation message strings. It is the same information a senior engineer would pass to a junior in a code review onboarding session — but expressed once, as a system prompt, and applied consistently on every review."

Q: "What if an agent makes a mistake?"

SAY:

"The pipeline is designed with that assumption. The reviewer agent in phase six is specifically checking whether the earlier implementer agents produced correct output. If the implementer agent missed a field, the reviewer catches it and returns `readyToCommit: false`. The developer sees the specific finding and fixes it before committing.

Think of it like a CI pipeline. The build does not fail because a developer is bad — it fails because a check caught something. The agent pipeline is the same pattern."

Q: "Is this all running locally or in the cloud?"

SAY:

"The agent orchestration runs in Claude Code — which runs locally on the developer machine. The underlying model calls go to Anthropic's API. The AI analysis in the SupportIQ platform at runtime goes to EPAM DIAL — our internal enterprise AI gateway. So the agent coordination is local, the model inference is cloud, and the data never leaves our infrastructure boundaries."

Q: "Can this workflow be reused for other projects?"

SAY:

"Yes — that is the design intent. The workflow scripts are in the `.claude/workflows` directory and are parameterised. The feature-implementation workflow takes an issue number as an argument. The code-review workflow takes a branch name. Any Spring Boot project that follows similar conventions could adopt these workflows with minor prompt adjustments. The orchestration pattern is generic — only the convention rules in the agent prompts are project-specific."

Q: "How long does one full feature implementation cycle take?"

SAY:

"The six-phase workflow — from reading the GitHub issue to a review-approved implementation — takes approximately three to five minutes for a standard feature. The parallel phases bring the total down significantly. Without parallelism, running nine agents sequentially would take fifteen to twenty minutes. The parallel design cuts that to under five."

FINAL NOTES

- **This audience is technical.** They will follow the flow diagrams. Do not rush past them.
 - **The JSON handoff diagram is your strongest moment.** Pause there. Let them see the data flow.
 - **If they ask to see the actual workflow code** — switch to VS Code and open `.claude/workflows/feature-implementation.js`. The code is clean and readable.
 - **"Multi-agent" does not mean magic.** Be precise — structured data handoffs, typed schemas, deterministic orchestration. Precision builds more credibility than hype.
-

Script prepared for post-demo technical Q&A — Multi-Agent SDLC, Day 8 AI-Native Engineering Series